Que dinámica tenemos que seguir para cada ejercicio

1. Abre `README.md` y ejecuta el proyecto.
2. Usa el catálogo: búsqueda, favorito y una nota. Guarda.
3. Entra en la Academia. Elige solo una pareja de ejemplo/evaluable.
4. Lee el objetivo. No tienes que hacer nada antes de abrir el código.
5. Ejecuta el ejemplo X. Después abre el archivo E del mismo número.
6. Sustituye el TODO por tu trabajo, sin cambiar la firma.
7. Ejecuta solo sus tests. Revisa primero el primer error.
8. Pulsa «Comprobar E##» en Academia: verde es correcto, rojo te dice qué falla y gris significa TODO sin resolver.
9. Comprueba la pantalla. Guarda un punto de progreso.

Las casillas son para recordar lo realizado, no para medir rapidez. Puedes trabajar con texto ampliado, por bloques cortos y sin agitar el dispositivo. No hay cuenta atrás en el cuestionario. Si necesitas retomar, escribe «último paso completado» y «siguiente acción» en una nota.

Nombres fijos: **X** = ejemplo resuelto; **E** = ejercicio evaluable. El rojo de un test pendiente no significa que hayas perdido el trabajo. Los requisitos obligatorios están en la lista de comprobación, no escondidos en las pistas.

PROMETEO
by thePower

E01 — Más equipos reales

1. Objetivo

Añadir dos clubes con al menos cinco jugadores verificados por club. No inventes posiciones ni estadísticas.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 01.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X01Catalogo.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E01Catalogo.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Abre `X01Catalogo.kt` y ejecuta su demo en Academia.
2. Implementa `addTeam` sin cambiar su firma.
3. Valida lote no vacío, un único club, posición admitida y fuente HTTPS.
4. Rechaza IDs repetidos y jugador repetido dentro del mismo club.
5. Añade dos clubes reales: prepara cinco fichas verificadas para cada uno.
6. Usa el formulario de Añadir equipos y comprueba su persistencia.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Lote vacío rechazado
- ID duplicado rechazado
- Dos clubes añadidos y persistidos
- Cada ficha nueva tiene fuente

9. Pruebas automatizadas

Clase: `E01Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E01Test'
```

También puedes pulsar el botón "Comprobar E01" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X01Catalogo`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E02 — Buscar, filtrar y ordenar

1. Objetivo

Combina texto, club, posición y favoritos; añade orden ascendente y descendente.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 02.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X02Filtros.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E02Filtros.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Ejecuta la búsqueda de `X02Filtros` con y sin tildes.
2. Normaliza el texto de búsqueda con `TextKey`.
3. Combina las condiciones con AND: texto, club, posición y favoritos.
4. Ordena por nombre normalizado y aplica el sentido solicitado.
5. Activa el control E02 de la pantalla para probar tu función.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- MBAPPÉ y mbappe encuentran el mismo jugador
- Filtros combinados respetados
- Lista vacía controlada
- Orden inverso correcto

9. Pruebas automatizadas

Clase: `E02Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E02Test'
```

También puedes pulsar el botón "Comprobar E02" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X02Filtros`; no copies su resultado sin adaptar el contrato.

- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E03 — Etiquetas y valoración personal

1. Objetivo

Guarda hasta cinco etiquetas únicas y una valoración personal de 0 a 5; nunca la presentes como estadística oficial.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 03.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X03Etiquetas.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E03Etiquetas.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Consulta cómo `X03Etiquetas` cambia una propiedad sin perder las demás.
2. Recorta etiquetas y elimina vacías.
3. Elimina duplicados sin distinguir tildes o mayúsculas.
4. Rechaza más de cinco etiquetas o etiquetas de más de 24 caracteres.
5. Valida valoración 0–5 y conserva favorito y nota.
6. Prueba guardar y volver a abrir la ficha.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Etiquetas equivalentes no se repiten
- Se conservan nota y favorito
- Valor 6 rechazado
- Persistencia comprobada

8bis. Guardado básico incluido en la app

La app ya guarda etiquetas y valoración con el botón "Guardar etiquetas (básico)": separa por comas, recorta espacios, descarta vacías y acepta la valoración que pueda leerse (si no es un número, 0). Ese guardado no aplica las reglas del contrato (etiquetas únicas, máximo cinco, valoración de 0 a 5) ni lanza las excepciones de las pruebas: para eso está tu E03. Si tu E03 está bien hecho, su botón "E03: guardar etiquetas y valoración" debe fallar en los casos prohibidos y el básico debe seguir guardando siempre.

9. Pruebas automatizadas

Clase: `E03Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E03Test'
```

También puedes pulsar el botón "Comprobar E03" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X03Etiquetas`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E04 — Comparar fichas sin inventar datos

1. Objetivo

Compara entre dos y cuatro jugadores distintos mostrando nombre, club y posición. No inventes quién es mejor.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 04.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X04Comparador.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E04Comparador.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Lee `X04Comparador` y localiza los campos reales disponibles.
2. Exige de dos a cuatro fichas con IDs distintos.
3. Devuelve una línea por jugador: `nombre | club | posición`.
4. Prueba la pantalla Comparar jugadores.
5. No añadas un ganador ni supuestas habilidades deportivas.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Dos y cuatro fichas válidas
- Una ficha rechazada
- Duplicados rechazados
- No se inventan estadísticas

9. Pruebas automatizadas

Clase: `E04Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E04Test'
```

También puedes pulsar el botón "Comprobar E04" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X04Comparador`; no copies su resultado sin adaptar el contrato.

- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E05 — Sorpresa respetando filtros

1. Objetivo

Elige solo entre candidatos visibles; controla lista vacía y usa un índice para que la regla sea comprobable.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 05.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X05Sensor.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E05Sensor.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Consulta `X05Sensor` y el evento del acelerómetro en `MainActivity`.
2. Si no hay candidatos devuelve `null`.
3. Convierte el índice recibido a una posición válida, incluso si es negativo.
4. Comprueba que eliges de la lista recibida, sin volver al catálogo global.
5. Activa agitar y combina filtros; también puedes usar el botón E05 sin sensor.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Vacío devuelve null
- Índice negativo válido
- Resultado pertenece a candidatos
- Sensor se libera al pausar

9. Pruebas automatizadas

Clase: `E05Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E05Test'
```

También puedes pulsar el botón "Comprobar E05" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X05Sensor`; no copies su resultado sin adaptar el contrato.

- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E06 — Filtro de contraste reversible

1. Objetivo

Valida un contraste entre 0 y 3. La pantalla aplicará tu regla a una copia conservando el original.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 06.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X06Contraste.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E06Contraste.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Ejecuta `X06Contraste` y estudia `MediaArt.gray`.
2. Valida un valor finito entre 0 y 3.
3. Devuelve `escala=value` y `desplazamiento=128×(1-value)`.
4. La interfaz ya aplica los parámetros sobre el original; no cambies el original.
5. Prueba 0, 1, 1.4 y recuperar original.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Contraste 1 no cambia el nivel
- NaN y fuera de rango rechazados
- Original recuperable
- Filtro visible

9. Pruebas automatizadas

Clase: `E06Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E06Test'
```

También puedes pulsar el botón "Comprobar E06" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X06Contraste`; no copies su resultado sin adaptar el contrato.

- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E07 — Imagen persistente por jugador

1. Objetivo

Define rutas privadas seguras por ID; conecta captura/importación con recuperación tras reinicio.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 07.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X07Imagen.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E07Imagen.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Consulta `X07Imagen` y observa el directorio privado `filesDir`.
2. Exige un ID positivo y una extensión png o jpg.
3. Devuelve `jugador_ID.extension` sin aceptar rutas aportadas por el usuario.
4. Importa una imagen propia y pulsa conservar imagen.
5. Cierra/reabre la app y comprueba que solo cambia la ficha elegida.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Ruta no manipulable
- Formato inválido rechazado
- Imagen reaparece al reiniciar
- Otra ficha no cambia

8bis. Guardado básico incluido en la app

La app ya conserva tu imagen con el botón "Guardar imagen propia (básico)": la guarda como `imagen_ID.png` en el almacenamiento interno y la recarga al reabrir la ficha. Ese guardado no valida ID ni extensión ni lanza las excepciones de las pruebas: para eso está tu E07. Si tu E07 está bien hecho, su botón debe rechazar los casos prohibidos y el básico debe seguir guardando siempre.

9. Pruebas automatizadas

Clase: `E07Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E07Test'
```

También puedes pulsar el botón "Comprobar E07" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.

- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X07Imagen`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E08 — Playlist y comentarios de voz

1. Objetivo

Calcula siguiente/anterior sin salir de rango. Integra con el reproductor y los audios grabados del jugador.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 08.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X08Audio.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E08Audio.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Lee `X08Audio` y las funciones de reproducción/grabación de `MainActivity`.
2. Admite solo dirección -1 o 1.
3. Devuelve `null` si no hay elementos.
4. Valida el índice actual y calcula anterior/siguiente con recorrido circular.
5. Graba dos comentarios de una ficha y comprueba su playlist.
6. Verifica que al salir se descarta una grabación aún no guardada.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Anterior desde 0 vuelve al último
- Vacío devuelve null
- Error de medio controlado
- Permiso rechazado no cierra app

9. Pruebas automatizadas

Clase: `E08Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E08Test'
```

También puedes pulsar el botón "Comprobar E08" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X08Audio`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E09 — Once inicial 4-3-3

1. Objetivo

Valida once jugadores distintos y reparto 1 portero, 4 defensas, 3 centrocampistas y 3 delanteros. Implementa sustitución visual.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 09.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X09Alineacion.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E09Alineacion.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Ejecuta el trío de `X09Alineacion`.
2. Devuelve `false` si no hay once IDs distintos.
3. Cuenta posiciones y exige exactamente 1/4/3/3.
4. Usa los espacios ya preparados en la pantalla para sustituir/quitar.
5. Carga o añade los jugadores que falten para completar las posiciones.
6. Distingue guardar un borrador de validar un once completo.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Once correcto aceptado
- Duplicado rechazado
- Reparto incorrecto rechazado
- Borrador incompleto guardado

9. Pruebas automatizadas

Clase: `E09Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E09Test'
```

También puedes pulsar el botón "Comprobar E09" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X09Alineacion`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E10 — Colecciones múltiples

1. Objetivo

Crea nombres únicos ignorando tildes/mayúsculas y espacios. Vincula jugadores sin duplicarlos.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 10.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X10Colecciones.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E10Colecciones.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Consulta `X10Colecciones`: un integrante no se repite.
2. Recorta el nombre y exige de 1 a 40 caracteres.
3. Rechaza un nombre equivalente a otro al normalizarlo.
4. Devuelve la lista previa más el nombre nuevo limpio.
5. Usa la pantalla para crear, renombrar, editar integrantes y eliminar.
6. Comprueba que borrar una colección no borra fichas ni otras colecciones.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Nombre vacío rechazado
- Nombre equivalente rechazado
- Integrantes sin duplicados
- Borrado independiente

9. Pruebas automatizadas

Clase: `E10Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E10Test'
```

También puedes pulsar el botón "Comprobar E10" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X10Colecciones`; no copies su resultado sin adaptar el contrato.

- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E11 — Copia de seguridad validada

1. Objetivo

Exporta y restaura favoritos/notas; valida antes de escribir. Conserva los jugadores ausentes de la copia.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 11.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X11Copia.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E11Copia.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Lee `X11Copia` y `Wire`: el formato es JSON versión 1.
2. Valida toda la copia antes de mezclar o guardar.
3. Usa solo identificadores conocidos.
4. La copia sobrescribe favorito/nota de IDs presentes; conserva los ausentes.
5. Prueba exportar, cambiar una nota e importar con confirmación.
6. No amplíes silenciosamente la copia a audios o imágenes: no están en este contrato.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Copia válida recuperada
- JSON inválido no escribe
- Cancelación no escribe
- Datos ausentes conservados

9. Pruebas automatizadas

Clase: `E11Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E11Test'
```

También puedes pulsar el botón "Comprobar E11" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X11Copia`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E12 — Datos guardados sin conexión

1. Objetivo

Distingue respuesta válida vacía de error. Mantén último resultado al fallar la descarga.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 12.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X12Cache.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E12Cache.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Consulta `X12Cache` y el tipo `CacheResult`.
2. Si `remote` no es null y no hay error, úsalo como dato actual, incluso si está vacío.
3. Si falla la descarga, usa `saved` y marca `stale=true`.
4. Distingue mensaje sin datos de mensaje datos guardados.
5. Prueba con respuestas controladas en tests, no con la disponibilidad de una web.
6. La pantalla de red usa un catálogo no vacío; no confundas su regla de producto con el contrato del helper.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Error conserva cache
- Remoto vacío válido en helper
- Primera apertura sin datos controlada
- Fecha de actualización visible al actualizar

9. Pruebas automatizadas

Clase: `E12Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E12Test'
```

También puedes pulsar el botón "Comprobar E12" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X12Cache`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E13 — Reto de cinco preguntas

1. Objetivo

Crea cinco preguntas de club sin jugadores repetidos, dos opciones distintas y exactamente una correcta.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 13.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X13Quiz.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E13Quiz.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Ejecuta la pregunta simple de `X13Quiz`.
2. Rechaza cantidad no positiva, IDs repetidos, jugadores insuficientes o menos de dos clubes.
3. Construye `count` preguntas sin repetir jugador.
4. Cada pregunta tendrá dos clubes distintos y exactamente una respuesta correcta.
5. La pantalla ya controla una respuesta por pregunta, puntuación y mejor resultado.
6. Prueba partida completa y pulsaciones repetidas; no hay límite de tiempo obligatorio.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Cinco jugadores distintos
- Opciones distintas y una correcta
- Doble pulsación no suma
- Mejor resultado conservado

9. Pruebas automatizadas

Clase: `E13Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E13Test'
```

También puedes pulsar el botón "Comprobar E13" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X13Quiz`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E14 — Diseñar y compartir cromo

1. Objetivo

Valida título de hasta 28 caracteres y uno de tres colores. Integra previsualización y compartir imagen.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 14.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X14Cromo.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E14Cromo.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Consulta `X14Cromo` y `MediaArt.card`.
2. Valida índice de color de 0 a 2.
3. Recorta el título y exige longitud de 1 a 28 caracteres.
4. Devuelve el título limpio sin alterar su contenido.
5. Previsualiza, cambia color y comparte PNG; comprueba desde otra aplicación.
6. No necesitas implementar de nuevo `CardProvider`: ya está preparado como infraestructura.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Título vacío rechazado
- Nombre largo encaja en cromo
- Previsualización correcta
- Otra app abre el PNG

9. Pruebas automatizadas

Clase: `E14Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E14Test'
```

También puedes pulsar el botón "Comprobar E14" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X14Cromo`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E15 — Intercambio con confirmación

1. Objetivo

Codifica colección con versión; valida tamaño e IDs, confirma recepción con ACK y permite cancelar.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 15.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X15Bluetooth.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E15Bluetooth.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Ejecuta el mensaje sencillo de `X15Bluetooth`.
2. Valida el máximo de 8192 bytes UTF-8 y versión 1.
3. Lee `ids` como enteros y exige lista no vacía, sin duplicados y conocida.
4. Devuelve los IDs en el orden recibido.
5. Usa dos equipos emparejados para enviar colección con ACK y recibirla.
6. Cancela la conexión y prueba datos inválidos; los tiempos están en `BluetoothLink`.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Versión incorrecta rechazada
- ID desconocido rechazado
- Recepción demostrada
- Cancelación y espera máxima

9. Pruebas automatizadas

Clase: `E15Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E15Test'
```

También puedes pulsar el botón "Comprobar E15" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X15Bluetooth`; no copies su resultado sin adaptar el contrato.
- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

PROMETEO
by thePower

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

E16 — Panel de mi colección

1. Objetivo

Agrupa por club sin duplicar IDs y muestra recuentos de tu colección, no estadísticas deportivas. Añade una ampliación propia.

2. Antes de empezar

- Ejecuta el catálogo básico. No tienes que resolver los otros 15 ejercicios primero.
- Haz una copia de tu trabajo o un commit.
- Abre Academia en el menú y busca el número 16.

3. Ejemplo resuelto que debes consultar

Archivo: `app/src/main/java/es/aula/futbol/ejemplos/X16Panel.kt`. Ejecuta su botón en Academia. Lee qué entra, qué se devuelve y cuándo se valida.

4. Archivo evaluable

`app/src/main/java/es/aula/futbol/evaluables/E16Panel.kt`. Mantén el objeto y la firma de la función. El TODO es intencional. La interfaz ya llama a esta función: no tienes que reconstruir toda la pantalla.

5. Pasos

1. Consulta el total de `X16Panel`.
2. Elimina duplicados por ID antes de contar.
3. Agrupa por club y devuelve un mapa ordenado por nombre de club.
4. Revisa el panel de favoritos con cero, uno y varios clubes.
5. Añade una ampliación propia acordada con el docente y documenta sus pruebas.

6. Punto de parada

Puedes parar cuando tu función compile y supere un caso sencillo. Guarda tu progreso antes de continuar con errores y límites. Los tiempos no se usan como criterio de evaluación.

7. No tienes que hacer

No cambies los IDs existentes, las firmas de los métodos ni el comportamiento de los otros ejercicios. No necesitas librerías nuevas para el helper. No inventes hechos deportivos. Las ampliaciones personales se acuerdan y se documentan aparte.

8. Lista de comprobación

- Vacío produce mapa vacío
- ID repetido cuenta una vez
- Recuento por club correcto
- Ampliación documentada

9. Pruebas automatizadas

Clase: `E16Test`. Prueba el ejemplo, el contrato y un caso límite. Comando para este ejercicio:

bash

```
./gradlew testDebugUnitTest --tests 'es.aula.futbol.E16Test'
```

También puedes pulsar el botón "Comprobar E16" dentro de la app (Academia): ejecuta los mismos contratos y te dice exactamente qué caso falla. En Windows usa `gradlew.bat`. Un test rojo por `NotImplementedError` significa que aún falta tu implementación, no que el proyecto base deba dejar de abrirse.

10. Entrega

- El `.kt` evaluable con tu solución y comentarios propios.
- Una captura del caso correcto y otra de una validación.
- Resultado real de pruebas, indicando qué no pudiste ejecutar.
- Cinco líneas: qué has hecho, qué error encontraste y cómo lo comprobaste.

11. Pistas progresivas

- **Pista 1:** identifica tipos de parámetros y retorno antes de escribir.
- **Pista 2:** localiza la técnica equivalente en `X16Panel`; no copies su resultado sin adaptar el contrato.

- **Pista 3:** resuelve un caso válido y después añade las validaciones de la lista.

12. Corrección observable (10 puntos)

Criterio	Puntos
Contrato y caso normal	4
Validaciones/límites	2
Integración visible	2
Pruebas explicadas y legibilidad	2

Se admiten algoritmos distintos que cumplan el contrato; el estilo visual no sustituye la funcionalidad.

PROMETEO
by thePower